COS10022 Data Science Principles

Assignment 1 — Predictive Model Creation and Evaluation

The Workflow Explained

Design, every node, data decisions, results, insights,
troubleshooting and exploration

About this document

A study companion for the KNIME workflow `COS10022_A1_Workflow`, prepared with AI assistance (Claude). The numbers are from the executed run of 10 October 2026, seed 2026, checked independently by `verify_outputs.py` (result: PASS).

Use it to understand the work and to plan your report. Declare it in the genAI appendix, and write the report in your own words: Turnitin limits any single source to 5%, and your video has to explain the same reasoning naturally.

Sections follow the marking rubric. Section 11 maps each criterion to the pages that support it.

Contents

1 What was built **2**

2 Understanding the data and the cleaning decisions **2**

2.1 What the data looks like 2

2.2 Five facts found during exploration 2

2.3 Cleaning decisions — and why “no cleaning” was the right answer 3

3 Workflow design **3**

3.1 Design principles 3

3.2 Task 1 layout 4

3.3 Task 2 layout 4

4 Task 1 — node by node **4**

5 Task 2 — data preparation, node by node **6**

6 Model construction **7**

7 Results **8**

7.1 Task 1 8

7.2 Task 2 8

8 Comparison and insights **9**

9 Problem solving and troubleshooting **10**

10 Exploration and discovery in self-learning **10**

11 Rubric map **12**

A Metric formulas **12**

B Glossary **13**

1 What was built

- **One KNIME workflow, 28 nodes**, covering both tasks. Task 1 trains a logistic regression on `accident_2025_train2_73.csv` and tests it on `accident_2025_test2_73.txt`. Task 2 reads all four CSV files, builds a balanced 700-row training set and a 500-row test set, and compares logistic regression, Naïve Bayes and Random Forest on identical data.
- **Self-contained and portable**: the five input files are stored inside the workflow (`data/src/`). Every reader and writer uses *Relative to: Current workflow data area*, so the workflow runs unchanged on any computer. Outputs are written to `data/output/`.
- **Reproducible**: every random step (Partitioning, Shuffle, Random Forest) uses seed 2026, so a re-run gives identical results. The brief gives no mark if the re-run differs from the report.
- **Independently verified**: a separate Python script re-reads KNIME's output files and checks row counts, class balance, shuffling and the absence of extra columns. It also reproduces the Task 1 result exactly with a different implementation of logistic regression.
- **Submission package**: `COS10022_A1_submission.zip` holds the executed workflow (`.knwf`), the input files (`src/`), the output files (`output/`) and run instructions. The video still has to be added.

2 Understanding the data and the cleaning decisions

2.1 What the data looks like

Attribute	Type and range
Age	integer, 18–69
Gender	nominal: Male / Female
Speed_of_Impact	integer, 20–119
Helmet_Used, Seatbelt_Used	nominal: Yes / No
Survived (*target*)	nominal: Yes / No

File	Rows	Format	Note
`accident_2025_train1_73.csv`	420	comma	
`accident_2025_train2_73.csv`	420	comma	Task 1 training set
`accident_2025_test1_73.csv`	180	comma	
`accident_2025_test2_73.csv`	180	comma	same data as the .txt
`accident_2025_test2_73.txt`	180	**tab**, CRLF	Task 1 test set

2.2 Five facts found during exploration

1. **The four CSVs are two different 70/30 splits of the same 600 records.** train1+test1 and train2+test2 contain the same records. Reading all four gives 1,200 rows, but only **599 are unique**: every record appears twice.
2. **Every file is sorted**: all "No" rows first, then all "Yes", each in descending speed.
3. **Class imbalance**: across the four CSVs, 378 Yes (31.5%) and 822 No (68.5%).
4. **A hard threshold**: nobody survives at a speed of 90 or more; below 90, about 45% survive.
5. **Only 189 unique "Yes" records exist** (and 410 unique "No").

2.3 Cleaning decisions — and why "no cleaning" was the right answer

The brief states the data is "cleaned and organised with no missing data". This was checked rather than assumed: no missing values, no out-of-range numbers, no misspelt categories. Each preprocessing step was then considered and deliberately rejected:

Decision	Justification
No missing-value handling	There are none, confirmed per column.
No normalisation	IRLS logistic regression does not need scaled inputs to converge; scaling would only change how the coefficients read. Naïve Bayes fits a distribution per attribute, and Random Forest only compares split thresholds, so neither is affected by scale.
No duplicate removal	Removing duplicates leaves **189 unique survivors, but the brief requires 350**. The task cannot be completed on de-duplicated data. The duplicates were kept as the brief requires, and their effect (train/test leakage) was *measured* instead (Section 8).
All five attributes used	The brief: "feeding all attributes into the model".
Survived kept nominal	Classifiers need a nominal target; a 0/1 number would be treated as a regression target.
Age and Speed kept numeric	They are measurements; logistic regression fits one slope per numeric attribute.

Why these facts drive the design

Sorted files mean random sampling and shuffling are essential. The imbalance means accuracy is a misleading metric. The duplicates mean the test set overlaps the training set. The speed threshold explains why a tree-based model has an advantage. Each later design choice comes back to one of these facts.

3 Workflow design

3.1 Design principles

The workflow follows Phase 4 of the Data Analytics Lifecycle (Model Building): prepare training and test data, build models, evaluate them. Six principles guided the layout:

1. **Separate stages**: read → prepare → learn → predict → evaluate, left to right, with no loops, so every number can be traced back to its source.
2. **Verify each stage** before the next uses it (row counts, column types, class counts), so loading errors cannot quietly reach the models.
3. **One training set and one test set feed all three models**, so differences in results come from the algorithms, not from the data.
4. **No test data ever reaches a learner**: the test Shuffle connects only to predictors.
5. **Reproducibility**: a fixed seed on every random node.
6. **Portability and evidence**: relative paths, and CSV Writers that save the intermediate sets and all predictions as output files.

3.2 Task 1 layout

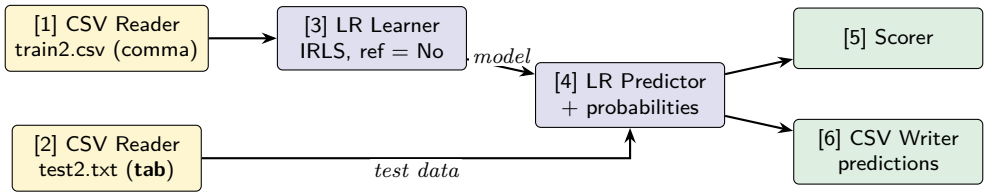


Figure 1: Task 1: one learner, one predictor, evaluated on the specified test file.

3.3 Task 2 layout

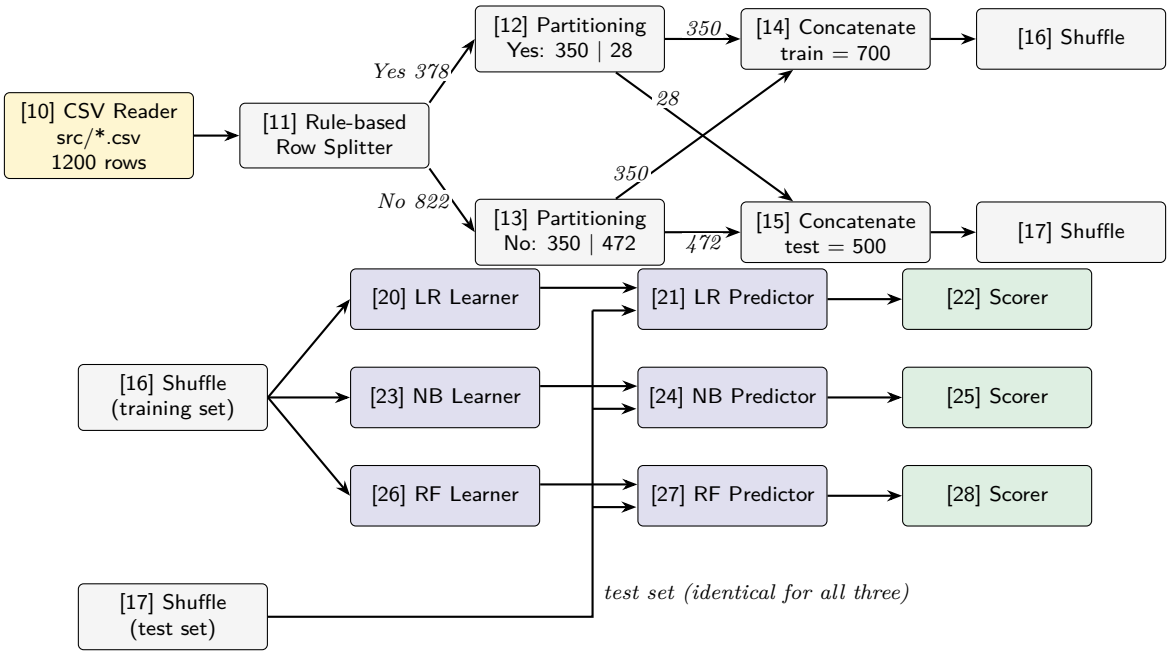


Figure 2: Task 2: class-wise sampling builds one training and one test set, shared by all three models. CSV Writers (not drawn) save the 700/500 sets and every model's predictions.

4 Task 1 — node by node

[1] CSV Reader — training file

Setting	Value	Reason
File	`src/accident_2025_train2_73.csv`	the training file named in the brief
Path mode	relative to workflow data area	runs on any computer
Delimiter / header	comma / first row	the file's actual format
Column types	auto-detected	Age, Speed integer; others string
Append path column	off	a path column would become a feature

Check: 420 rows, 6 columns.

[2] CSV Reader — test file (the loading problem)

Issue encountered: the test file is tab-separated

Symptom: with the default comma delimiter, every line of `accident_2025_test2_73.txt` lands in a single column, and the predictor cannot find Age, Speed and the other attributes.
Cause: the .txt uses **tab** separators and Windows (CRLF) line endings, while every other file uses commas.
Fix: column delimiter set to **Tab** (`\t`). The line-break row delimiter handles CRLF.
Confirmation: 180 rows, 6 columns, *with the same names and types as the training file.* The predictor requires the test table to match the columns the model was trained on.
Why not use test2.csv instead? It has the same data, but the brief names the .txt.

[3] Logistic Regression Learner

Theory. A linear score $z = w_0 + w_1x_1 + \cdots + w_kx_k$ is turned into a probability by the sigmoid, $P(\text{Yes}) = 1/(1 + e^{-z})$. Equivalently, $\log \frac{p}{1-p} = z$: each coefficient is the change in log-odds per unit of its input, and e^w is the odds ratio. Nominal inputs are dummy-coded against a base level. The decision boundary is linear, so a sharp cut-off (speed ≥ 90) can only be approximated by a smooth slope.

Setting	Value	Reason
Target	Survived	the label to predict
Features	all five	brief: all attributes
Reference category	**No**	coefficients then describe the odds of *surviving*; a positive sign means "raises survival odds"
Solver	**IRLS**	iteratively reweighted least squares gives the exact maximum-likelihood fit and is deterministic; ideal for 420 rows. SAG (stochastic average gradient) is approximate and depends on epochs, step size and seed; it is meant for large data.
Max epochs / ϵ	100 / 10^{-5}	stopping rules; IRLS converges in a few iterations
Coefficient statistics	on	standard errors and p-values show which features matter
Seed	2026	only SAG uses it; fixed for consistency

[4] Logistic Regression Predictor

Inputs: port 1 = model from [3], port 2 = test data from [2]. Class probabilities P(Yes)/P(No) appended, so confidence can be discussed, not just labels.

[5] Scorer and [6] CSV Writer

The Scorer compares `Survived` (actual) with `Prediction (Survived)` and reports the confusion matrix, accuracy, Cohen's kappa and per-class recall, precision, specificity and F-measure. The writer saves `output/task1_LR_predictions.csv`, overwriting on re-run so execution never fails.

5 Task 2 — data preparation, node by node

[10] CSV Reader — reading every CSV without touching `src`

Setting	Value	Reason
Mode	**Files in folder**: `src`	one node reads every file; the folder itself is never changed
File filter	extension = `csv`	includes the four CSVs, excludes the .txt (which is not a CSV)
Prepend file index to RowID	on	each file numbers its rows from Row0; without the prefix the RowIDs collide
Append path column	**off**	otherwise a "Path" column becomes a feature that reveals which file a row came from
Fail on differing specs	on	a file with a different structure stops the node instead of corrupting data

Check: 1,200 rows (4 files), 6 columns, no Path column.

[11] Rule-based Row Splitter

Rule: `$Survived$ = "Yes" => TRUE`. Rows where it is TRUE go to the top port, all others to the bottom.

- `$Survived$` refers to the column; `"Yes"` is a text constant; `=>` gives the outcome.
- **Result**: top = 378 Yes, bottom = 822 No; 378 + 822 = 1200 confirms no row was lost.
- **Why split by class first**: so exactly 350 can be drawn from *each* class. One random split of the whole table would give an unpredictable class mix.

How the splitter connects to the issues

Sorted data: each output is a block of rows in file order (descending speed), so the next node must sample randomly. **Duplicates**: both copies of each record land in the same output, which is where the later train/test overlap starts. **Exact text match**: `"yes"` or `"Yes "` would send every row to the bottom port. Checking 378/822 proves the rule works.

Workflow Coach — finding the partitioning node

The brief requires the Workflow Coach to be used *at one of the splitter's outputs* and the prompt to be reported. Click an output port of [11], ask for a node that splits a table into an exact number of rows, and record the exact wording and the suggestions (screenshot). A prompt following the week 2 structure states the *task* (split into two groups), the *constraint* (exactly 350 rows, the rest in the other group) and the *context* (the input is one class from the splitter). Justify choosing **Partitioning** over alternatives such as Row Sampling (which discards the remainder) or Row Splitter (rule-based, not count-based).

[12] and [13] Partitioning

Setting	Value	Reason
Mode	**Absolute, 350**	the brief requires *exactly* 350; a percentage cannot guarantee an exact count
Sampling	**Draw randomly**	the input is sorted by speed, so "take from top" would select the 350 fastest crashes, a biased sample
Seed	2026	the same 350 rows on every run

Yes branch [12]: top = 350 (training), bottom = 28 (test). No branch [13]: top = 350, bottom = 472.

[14] and [15] Concatenate

[14] stacks the two 350-row top outputs into the **700-row training set** (balanced 50/50). [15] stacks the remainders into the **500-row test set** (28 Yes, 472 No). Settings: union of columns; duplicate RowIDs get a suffix instead of failing. None occur here because of the file-index prefix, but this is the safe setting.

[16] and [17] Shuffle

After concatenation, the rows sit in blocks (all Yes, then all No). Shuffling removes that order, which matters for learners that process rows in sequence or in batches. The brief requires both sets to be shuffled. Seed 2026 keeps the order reproducible.

[18] and [19] CSV Writers

Save `task2_train_700.csv` and `task2_test_500.csv`: evidence of the exact counts, and the intermediate files the brief asks for.

6 Model construction

All three learners take the same training table [16]; all three predictors take the same test table [17].

[20]–[22] Logistic Regression

Same configuration as Task 1 (Section 4), so the two logistic regression results can be compared directly.

[23]–[25] Naïve Bayes

Theory. $P(c \mid a_1, \ldots, a_m) \propto P(c) \prod_j P(a_j \mid c)$. "Naïve" means the attributes are assumed independent given the class. Nominal attributes use frequency tables; numeric attributes (Age, Speed) use a Gaussian per class. $P(c)$, the **prior**, is learned from the training class proportions: here 50/50 by construction.
Settings: class column Survived; minimum standard deviation 10^{-4} (avoids division by zero); maximum 20 distinct nominal values (each attribute has 2); predictor appends class probabilities.

[26]–[28] Random Forest

Theory. An ensemble of decision trees. Each tree is trained on a bootstrap sample (rows drawn with replacement) and considers a random subset of attributes at each split; the forest predicts by majority vote. Trees split on thresholds, so "Speed < 90" can be learnt directly. Deep trees can memorise individual records.

Setting	Value	Reason
Number of trees	100	KNIME default; ample for five attributes
Split criterion	Information Gain Ratio	default; corrects information gain's bias towards attributes with many values
Attribute sampling	square root ($\sqrt{5} \approx 2$)	different trees see different attributes, which strengthens the ensemble
Row sampling	bootstrap, 100%	each tree learns from a different resample
Tree depth	unlimited	default (an overfitting risk discussed later)
Seed	**2026**	otherwise each run grows different trees

7 Results

7.1 Task 1

Training: 300 No / 120 Yes. Test: 111 No / 69 Yes.

	Predicted No	Predicted Yes
Actual No	92	19
Actual Yes	45	24

Accuracy	0.644
Always-"No" baseline	0.617
Cohen's kappa	0.190
Recall (Yes)	0.348
Precision (Yes)	0.558
Specificity	0.829

Logistic regression beats guessing "No" by only 2.7 percentage points and misses 65% of survivors. It leans towards "No" because 71% of its training rows were "No". This exact result was reproduced by an independent implementation of unpenalised logistic regression.

Coefficients (read them from node [3]'s coefficient table; the independent check gives these signs and sizes): Seatbelt $\approx +1.0$ (about 2.7× the odds of surviving), Helmet $\approx +0.8$ ($\approx$ 2.2×), Speed ≈ -0.035 per km/h ($\approx 0.70\times$ the odds per extra 10 km/h), Age ≈ 0 (no effect). These agree with common sense, which supports the model. Report which ones are statistically significant (p-values).

7.2 Task 2

Training: 350 Yes / 350 No. Test: 28 Yes / 472 No.

Model	Accuracy	Kappa	Recall Yes	Precision Yes	F_1 Yes	Specificity
Always "No"	**0.944**	0	0	–	–	1
Logistic Regression	0.706	0.131	0.714	0.126	0.214	0.706
Naïve Bayes	0.674	0.120	0.750	0.119	0.205	0.669
Random Forest	**0.752**	**0.222**	**0.929**	**0.176**	**0.295**	**0.742**

Table 1: Task 2 test-set results (positive class = Yes). Check them against your Scorer nodes.

Model	TN	FP	FN	TP
LR	333	139	8	20
NB	316	156	7	21
RF	350	122	2	26

8 Comparison and insights

1. **Every model is less accurate than always answering "No".** On a test set that is 94.4% "No", accuracy mostly measures agreement with the majority class. Kappa, F_1, and recall and precision for "Yes" show usefulness. The week 4 lecture makes the same point: accuracy alone does not establish a good model, and which error matters (Type I vs. Type II) depends on the situation. For accident survival, missing a survivor and wrongly predicting survival have very different costs.
2. **The balanced training set shifts every model towards "Yes".** The models learnt in a 50/50 world and were tested in a 94/6 world. They find most survivors (recall 0.71–0.93), but at least 8 out of 10 of their "survived" predictions are wrong (precision 0.12–0.18). Naïve Bayes is affected directly through its prior $P(\text{Yes}) = 0.5$, where the test rate is 0.056.
3. **The same algorithm, a different training mix: Task 1 vs. Task 2 logistic regression.** Recall rose from 0.35 to 0.71, and precision fell from 0.56 to 0.13. Nothing changed except the class balance of the training data. That is the clearest evidence of the prior effect.
4. **Random Forest wins, for two different reasons.**
   - *It suits the shape of the data*: survival drops to zero above speed 90. A tree learns that threshold directly; logistic regression can only fit a smooth slope in log-odds, and Naïve Bayes models speed as a bell curve per class.
   - *Leakage*: 222 of the 500 test rows (44%) are identical to a training row, because every record exists twice. Random Forest scores **0.793 on those rows and 0.719 on unseen rows**. Logistic regression (0.698 vs. 0.712) and Naïve Bayes (0.662 vs. 0.683) show no such gap, because they cannot memorise individual records. Part of Random Forest's lead is memorisation, and on genuinely unseen data it only roughly ties logistic regression.
5. **The "IID" training set, examined critically.** The brief calls the 700-row set IID (independent and identically distributed). Its class mix (50/50) matches neither the source data (31.5% Yes) nor the test set (5.6% Yes), and 44% of the test set duplicates it. The two sets are neither identically distributed nor independent. The design is good for learning what separates the classes, but it does not give an honest estimate of real-world performance.

Suggestions

- Remove duplicates *before* splitting. This would make the 350-per-class requirement impossible (189 unique survivors), so the sampling design itself would need to change, e.g. a stratified 70/30 split.
- Use k-fold cross-validation (X-Partitioner/X-Aggregator, week 4) rather than a single holdout split.
- Evaluate with kappa, F_1 or AUC, and choose the metric according to which error is more costly.
- Adjust the decision threshold, or correct the priors, to match the real-world class rate.
- Limit Random Forest depth to reduce memorisation; collect more informative attributes.

9 Problem solving and troubleshooting

Problems you are likely to meet when building this by hand, with symptoms and fixes:

Problem	Symptom	Fix
Tab-delimited .txt	test table has one column; predictor error	delimiter = Tab
Train/test types differ	predictor reports missing or mismatched columns	make both readers produce identical names and types
Folder reader includes the .txt	"differing specs" error or a garbage column	extension filter `csv`
Path column left on	a "Path" feature appears in the models	turn off "append path column"
Four readers + Concatenate	duplicate RowID error	file-index prefix, or Concatenate "append suffix"
Relative (%) partitioning	349 or 351 rows instead of 350	Absolute, 350
"Take from top" sampling	training set holds only high-speed crashes	Draw randomly
No seeds	numbers change each run; report no longer matches	seed on Partitioning, Shuffle, Random Forest
Splitter text typo (`"yes"`)	all rows in one output	exact `"Yes"`; check 378/822
Concatenate inputs swapped	training set of 378 or 822 rows	top outputs → train, bottom outputs → test
Predictor ports swapped	cannot connect, or no prediction column	model → port 1, data → port 2
Scorer columns wrong	empty or meaningless matrix	Survived vs. Prediction (Survived)
Target read as a number	learner refuses the target	keep Survived nominal
SAG solver	"did not converge" warning; results vary with seed	IRLS, or more epochs
Coefficient signs look backwards	"seatbelt lowers survival?"	check the reference category
Absolute file paths	fails on the tutor's computer	relative to workflow data area
Workflow Coach empty / K-AI unavailable	no suggestions	enable it in preferences and log in to KNIME Hub; K-AI has a monthly limit
Exporting without data	tutor's copy has no inputs or outputs	files in the data area; export the executed workflow

10 Exploration and discovery in self-learning

The Excellent band asks for independent exploration of tools *and* concepts, discoveries about workflow design, model behaviour and theory, experiments with multiple approaches, and critical reflection. Candidate explorations, each with the question, the method and what it shows:

Exploration	Question, method, finding
Leakage analysis	*Does the test set overlap the training set?* Match test rows to training rows (Joiner on all columns). Finding: 222/500 overlap; Random Forest 0.79 on seen vs. 0.72 on unseen rows; logistic regression and Naïve Bayes show no gap. Shows memorisation and why test data must be independent.
De-duplication paradox	*Can the task run on clean data?* Count unique records per class (Duplicate Row Filter + GroupBy). Finding: 189 unique survivors, below the 350 required. The design depends on duplicates.
Speed threshold	*Which attribute drives survival?* Survival rate by speed band (Binner + GroupBy, or a bar chart). Finding: 0% above 90. Links data understanding to model choice.
Training mix effect	*Why does the same algorithm behave differently?* Compare Task 1 and Task 2 logistic regression. Finding: recall 0.35 → 0.71, precision 0.56 → 0.13. The prior effect.
Seed sensitivity	*How reliable is one split?* Re-run Task 2 with 2–3 seeds and report the spread. Motivates cross-validation.
IRLS vs. SAG	*Does the optimiser matter?* Compare coefficients and accuracy. Exact vs. approximate optimisation.
Limiting tree depth	*Is Random Forest memorising?* Limit depth and compare seen vs. unseen accuracy.

Writing the reflection. Be specific: what surprised you (models below the always-"No" baseline; Random Forest's lead partly from memorisation; a "clean" dataset that cannot be de-duplicated), which theory explained it (priors, leakage, linear vs. threshold decision boundaries), and what you would do differently (de-duplicate first, cross-validate, choose metrics by error cost).

11 Rubric map

Criterion	Weight	Covered in
Understanding of workflow design	5%	Section 3 (principles, diagrams)
Detailed node configuration	15%	Sections 4, 5, 6 (every setting with its reason)
Data loading and preprocessing	10%	Section 2, node [2] issue, node [10]
Rule-based Row Splitter	5%	node [11] and its issue box
Dataset partitioning	10%	nodes [12]–[17]
Workflow Coach and node selection	5%	Workflow Coach subsection (your own prompt and screenshot)
Model construction	5%	Section 6
Model evaluation and results	10%	Section 7
Comparison and insights	10%	Section 8
Problem solving and troubleshooting	5%	node [2] and [11] issues; Section 9
Report quality and presentation	10%	numbered, captioned figures and tables; consistent terms
Exploration and self-learning	10%	Section 10

Before you submit

- Header on page 1: name, student ID, unit code, class, tutor. At most 10 pages. Questions in the brief's order.
- Your own Workflow Coach prompt and screenshot; your own screenshots of every node dialog and Scorer.
- Re-import the submitted `.knwf`, run it, and confirm it matches the report.
- Video inside the zip. GenAI declaration appendix completed. Turnitin below 25% overall and below 5% per source.

A Metric formulas

With positive class = Yes:

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN} \qquad \text{Recall} = \frac{TP}{TP+FN} \qquad \text{Precision} = \frac{TP}{TP+FP}$$

$$\text{Specificity} = \frac{TN}{TN+FP} \qquad F_1 = \frac{2PR}{P+R} \qquad \kappa = \frac{p_o - p_e}{1-p_e}$$

where p_o is the observed accuracy and p_e the accuracy expected by chance from the row and column totals. Worked example (Random Forest): $p_o = 0.752$; $p_e = 0.056 \times 0.296 + 0.944 \times 0.704 = 0.681$; $\kappa = (0.752 - 0.681)/(1 - 0.681) = 0.222$.

B Glossary

Holdout split	dividing data once into training and test sets
IID	independent and identically distributed: each record drawn independently from the same distribution
Data leakage	information from the test set present in training, inflating test scores
Prior	a class's probability before seeing the attributes; learnt from the training proportions
Bootstrap sample	a resample of the same size drawn with replacement
Reference category	the class the logistic regression coefficients are measured against
Majority-class baseline	the score of always predicting the most common class
Cohen's kappa	agreement between predictions and truth beyond what chance would give